

Databricks Master Data Engineer

6-Month Exhaustive Training & Certification Program

Advanced Architecture, Hands-on Labs, and MLOps

Abstract

This 24-week program is designed to transform you from a practitioner to a Master Data Engineer on the Databricks Data Intelligence Platform. It progressively builds from lakehouse and Delta Lake foundations to advanced Unity Catalog governance, declarative pipelines, structured streaming, Mosaic AI, and production-grade CI/CD, integrating daily theory, weekly service-specific mini-projects, architectural use case analyses, and massive monthly Capstone projects. A tracking timetable is included at the end.

Contents

1 Milestone 1: Lakehouse Foundations (Month 1)	2
2 Milestone 2: Warehousing & Analytics (Month 2)	6
3 Milestone 3: Batch ETL & Orchestration (Month 3)	9
4 Milestone 4: Streaming & Real-Time (Month 4)	13
5 Milestone 5: ML & MLOps (Month 5)	17
6 Milestone 6: Security, Production & Certification (Month 6)	21
7 Appendix A: Glossary	24
8 Appendix B: Assessment & Grading Model	26
9 Appendix C: Interview Preparation	27
10 Appendix D: Self-Check Answer Key	28
11 Appendix E: Datasets & Project Data	32
12 Appendix F: Portfolio & Presentation Guide	33
13 6-Month Progress Tracking Timetable	35

Prerequisites & How to Use This Syllabus

- **Prerequisite skills:** intermediate Python (PySpark helpful), solid SQL, Git workflows (branching, pull requests), Linux/CLI basics, and REST API fundamentals.
- **Weekly cadence:** approximately 10–12 hours per week — Monday to Wednesday for theory, Thursday for the hands-on project, Friday for the architecture use case.
- **Deliverable expectation:** every capstone must produce a git repository, an architecture diagram, a cost estimate (in DBUs and cloud infrastructure), and a security review checklist, per the Acceptance Criteria boxes that follow each capstone.
- **Assessment:** the 24-week tracking timetable at the end of this document doubles as the assessment checklist — check off each week only when its project and deliverables are complete.

Environment setup checklist:

- Install Python 3.x, Git, Docker Desktop, and VS Code (with the Databricks extension).
- Install the Databricks CLI v0.200+ (`pip install databricks-cli` or the native installer) and `databricks-sdk-py`; verify with `databricks auth profiles` and `databricks workspace list /`.
- Create a free Databricks trial workspace (or use Databricks Community Edition / an existing organizational workspace) with guardrails: a dedicated catalog/schema for labs, cluster auto-termination set to 30 minutes or less, and a budget alert on the cloud account.
- Clone the course repository structure: `data/`, `src/`, `tests/`, `bundles/`, `docs/`.
- Configure credentials via a CLI profile with OAuth (`databricks auth login`) or a personal access token stored in an environment variable — never hardcode tokens in code or notebooks.

1 Milestone 1: Lakehouse Foundations (Month 1)

Objective: Master the Databricks workspace, Delta Lake storage, the medallion architecture, and Apache Spark fundamentals.

Learning outcomes: navigate and administer a Databricks workspace, explain Delta Lake's transaction log and ACID guarantees, design Bronze/Silver/Gold data flows, and write idiomatic PySpark against managed Delta tables.

Week 1: Databricks Platform & Workspace Fundamentals

Outcomes: explain the Databricks control plane vs. data plane split, navigate workspaces, notebooks, and repos, choose between all-purpose and jobs compute, and authenticate the Databricks CLI and Python SDK.

- **Monday:** Platform architecture: control plane, data plane, serverless vs. classic compute, workspace objects.
- **Tuesday:** Notebooks, Repos (Git folders), magic commands, and `dbutils` utilities.
- **Wednesday:** Compute: all-purpose clusters, job clusters, policies, pools, and auto-termination; the Databricks CLI and `databricks-sdk-py`.

- **Thursday: Hands-on Project:** Provision a trial workspace, configure the Databricks CLI with an OAuth profile, create an all-purpose cluster with 30-minute auto-termination, clone a Git repo into a Git folder, and run a smoke-test notebook via `databricks jobs submit` using the SDK.
- **Friday: Use Case & Architecture:** Design the workspace and compute layout for a 200-person data organization spanning three business units with dev/staging/prod isolation.

Resources: Databricks documentation (<https://docs.databricks.com/>); Data Engineering Zoomcamp (<https://github.com/DataTalksClub/data-engineering-zoomcamp>). **Self-check:**

- Which resources live in the control plane and which in the data plane?
- Why are job clusters cheaper than all-purpose clusters for scheduled pipelines?
- What does a cluster policy restrict, and who benefits from that restriction?
- How does a Git folder (Repo) differ from a workspace notebook folder?
- Why prefer OAuth (`databricks auth login`) over a long-lived personal access token?

Production Note: *Set auto-termination on every non-serverless cluster before your first real workload: an idle all-purpose cluster left running over a weekend is the single most common first-month bill shock. Enforce it with a cluster policy that caps `autotermination_minutes` and deny users the `unpinned` permission on shared clusters.*

Week 2: Delta Lake Foundations

Outcomes: explain the Delta transaction log, ACID semantics, and schema enforcement/evolution, use time travel and `DESCRIBE HISTORY`, and recover from accidental writes with `RESTORE`.

- **Monday:** Delta Lake architecture: Parquet data files, the `_delta_log` transaction log, and atomic commits.
- **Tuesday:** ACID guarantees, optimistic concurrency, isolation levels, and `MERGE INTO` semantics.
- **Wednesday:** Schema enforcement and evolution, constraints, generated columns, time travel, and `RESTORE`.
- **Thursday: Hands-on Project:** Create Delta tables from CSV seed data, force a schema mismatch and observe enforcement, evolve the schema deliberately with `mergeSchema`, delete a partition “by accident,” then recover it with time travel and `RESTORE`; document every step from `DESCRIBE HISTORY`.
- **Friday: Use Case & Architecture:** Design the landing and recovery strategy for a financial-services lake that must prove to auditors exactly what the data looked like at any point in the last 90 days.

Resources: Delta Lake documentation (<https://docs.delta.io/>); Delta Lake VLDB paper (<https://www.vldb.org/pvldb/vol13/p3411-armbrust.pdf>). **Self-check:**

- What makes a Delta commit atomic even though it spans many Parquet files?
- Which isolation level does Delta use for concurrent writes to one table?
- `mergeSchema` vs. `overwriteSchema`: when is each appropriate?

- What does `VACUUM` permanently destroy, and why does the 7-day default exist?
- Why is `MERGE INTO` preferred over delete-then-insert for upserts?

Production Note: *Time travel is a recovery tool with an expiry date: `VACUUM` removes the data files that older versions reference. Never lower retention below your longest recovery objective, and treat any `RETAIN` under 168 hours as a change-control event requiring sign-off.*

Week 3: The Medallion Architecture (Bronze/Silver/Gold)

Outcomes: justify the Bronze/Silver/Gold layering, design data-quality gates between layers, model a Gold star schema, and name catalogs/schemas/tables for a multi-team lakehouse.

- **Monday:** Medallion rationale: progressive refinement, idempotent reprocessing, and the cost of skipping layers.
- **Tuesday:** Bronze patterns: append-only raw capture, metadata columns (`_ingested_at`, `_source_file`), and schema-on-read trade-offs.
- **Wednesday:** Silver cleansing and conformance, Gold dimensional modeling, and Unity Catalog naming (`catalog.schema.table`).
- **Thursday: Hands-on Project:** Build a three-layer medallion flow for the Olist e-commerce dataset: land raw CSVs into Bronze with ingestion metadata, cleanse and deduplicate into Silver with quarantine for rejected rows, and publish a Gold `fact_orders` plus two dimensions; reconcile row counts across layers.
- **Friday: Use Case & Architecture:** Architect the medallion lakehouse for a retail chain that must reprocess two years of history after a business-rule change without double-counting revenue.

Resources: Databricks medallion architecture guide (<https://www.databricks.com/glossary/medallion-architecture>); Data Engineer Handbook (<https://github.com/DataExpert-io/data-engineer->

Self-check:

- Why keep Bronze append-only even when storage is “cheap”?
- What belongs in a quarantine table instead of being silently dropped?
- How does the medallion layout make a re-run idempotent?
- Why model Gold as a star schema rather than a wide denormalized table?
- Which metadata columns prove lineage from Gold back to the raw file?

Week 4: Apache Spark on Databricks

Outcomes: explain Spark’s driver/executor model and lazy evaluation, read the query plan from the Spark UI, use DataFrame transformations vs. actions, and partition data deliberately with repartition and coalesce.

- **Monday:** Spark architecture: driver, executors, DAG, stages, tasks, and lazy evaluation.
- **Tuesday:** DataFrame API: transformations vs. actions, Catalyst optimizer, and the query plan (`explain`).
- **Wednesday:** Partitions, shuffles, skew, caching, and the Spark UI on Databricks.

- **Thursday: Hands-on Project:** Write a PySpark notebook that ingests 10M NYC Taxi rows, runs aggregations with deliberate narrow and wide transformations, then use the Spark UI to find the shuffle stage, identify skewed partitions, and fix them with salting or `repartition`.
- **Friday: Use Case & Architecture:** Diagnose a nightly job that grew from 40 minutes to 6 hours as data grew 10x: produce a tuning plan grounded in the Spark UI evidence you would collect.

Resources: Apache Spark documentation (<https://spark.apache.org/docs/latest/>); Databricks runtime release notes (<https://docs.databricks.com/release-notes/runtime/>). **Self-check:**

- Why does lazy evaluation let Catalyst optimize your job?
- What triggers a shuffle, and why is it expensive?
- `repartition` vs. `coalesce`: which can increase partition count?
- What does a long tail in the Spark UI stage summary usually indicate?
- When does `cache()` hurt more than it helps?

CAPSTONE PROJECT - Milestone 1: Build a Retail Medallion Lakehouse. Ingest the Olist dataset into a Bronze schema with ingestion metadata, cleanse and deduplicate into Silver with a quarantine table and row-count reconciliation, publish a Gold star schema (`fact_orders`, `dim_customer`, `dim_product`, `dim_date`), and demonstrate time-travel recovery of a deliberately corrupted Silver table.

Acceptance Criteria

- End-to-end data pipeline
- Architecture diagram and data model
- At least 10% data quality improvement
- DESCRIBE table command
- Month-over-month performance optimization
- Security audit for dim_customer

Milestone 1 Capstone: Reference Solution Sketch

- **Architecture:** volume-landed CSVs → Bronze (append-only Delta with `_ingested_at`) → Silver (typed, deduplicated, quarantined rejects) → Gold star schema — decouples raw evidence from curated analytics.
- **Steps:** create catalog training with schemas `bronze`, `silver`, `gold`; upload Olist CSVs to a Unity Catalog volume; write parameterized PySpark notebooks per layer; add reconciliation notebook asserting `accepted + rejected = input`; corrupt Silver with a bad `DELETE`; recover with `RESTORE TO VERSION AS OF`.
- **Hardest part — idempotent Silver merge:**

```
MERGE INTO silver.orders t
USING bronze_orders_deduped s
ON t.order_id = s.order_id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
```

- **Pitfalls:** running `VACUUM` before the recovery demo destroys the versions you need; a non-parameterized notebook forces code edits per re-run; forgetting to stop the all-purpose cluster leaks DBUs overnight.

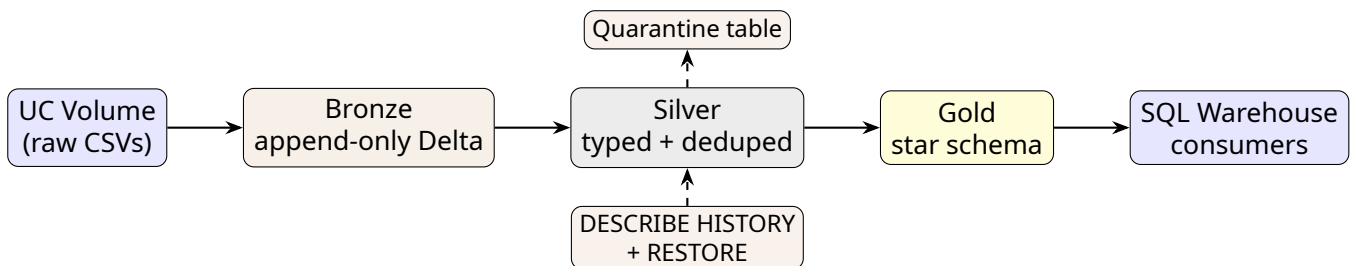


Figure 1: Milestone 1 reference architecture: medallion lakehouse with quality gates and time-travel recovery

2 Milestone 2: Warehousing & Analytics (Month 2)

Objective: Optimize Delta storage, serve SQL analytics with warehouses, and govern data with Unity Catalog.

Learning outcomes: compact and cluster Delta tables for selective queries, operate SQL warehouses (serverless and classic), build AI/BI dashboards and Genie spaces, and implement catalog-level governance with Unity Catalog.

Week 5: Delta Lake Optimization (OPTIMIZE, Z-ORDER, Liquid Clustering)

Outcomes: diagnose the small-file problem, run `OPTIMIZE` with `ZORDER BY`, explain liquid clustering and when it supersedes partitioning/Z-ORDER, and schedule `VACUUM` safely.

- **Monday:** File layout and data skipping: file-level statistics, the small-file problem, and bloom filters.
- **Tuesday:** `OPTIMIZE` compaction, `ZORDER BY` multi-dimensional clustering, and auto-optimize vs. scheduled jobs.
- **Wednesday:** Liquid clustering (`CLUSTER BY`), predictive optimization, deletion vectors, and `VACUUM` retention.
- **Thursday: Hands-on Project:** Create a 50M-row events table with thousands of small files, benchmark a selective query, then compact with `OPTIMIZE ZORDER BY`, convert a second copy to liquid clustering, benchmark again, and record file counts and timings in a results table.
- **Friday: Use Case & Architecture:** Remediate a lakehouse whose nightly streaming sink produced 2 million tiny files: design the compaction, clustering, and write-cadence plan, and defend liquid clustering vs. partition + Z-ORDER.

Resources: Delta Lake optimization documentation (<https://docs.databricks.com/delta/optimize.html>); liquid clustering announcement and docs (<https://docs.databricks.com/delta/clustering.html>). **Self-check:**

- Why do many small files slow queries even when total bytes are unchanged?

- Which columns make good ZORDER BY candidates?
- What does liquid clustering let you change without rewriting the table?
- What do deletion vectors avoid rewriting on DELETE?
- Why should OPTIMIZE not run against the table a stream is writing at the same moment?

Production Note: *Prefer liquid clustering for new large tables: it avoids the partition-column decision you will regret in year two, allows CLUSTER BY changes without a full rewrite, and composes with predictive optimization so maintenance runs itself on serverless. Keep ZORDER for legacy tables you cannot yet migrate.*

Week 6: Databricks SQL & SQL Warehouses

Outcomes: size and operate SQL warehouses (serverless, pro, classic), write warehouse-grade SQL against Unity Catalog tables, use materialized views and result caching, and control concurrency and cost.

- **Monday:** Databricks SQL architecture: SQL warehouses, serverless vs. pro vs. classic, Photon, and the query profile.
- **Tuesday:** Analytical SQL: window functions, Delta time travel in SQL, QUALIFY, and query federation basics.
- **Wednesday:** Materialized views and streaming tables in DBSQL, result caching, warehouse scaling, and monitoring with system tables.
- **Thursday: Hands-on Project:** Load the Gold star schema from Month 1 behind a serverless SQL warehouse, build a materialized view for daily revenue, capture the query profile before/after, and configure min/max clusters plus auto-stop; document cost per query from `system.billing.usage`.
- **Friday: Use Case & Architecture:** Design the warehouse topology for a company where 500 analysts run morning dashboards while data engineers run backfills on the same tables without contention.

Resources: Databricks SQL documentation (<https://docs.databricks.com/sql/>); Photon engine overview (<https://www.databricks.com/product/photon>). **Self-check:**

- What does Photon accelerate and why does it matter for SQL workloads?
- When does a materialized view beat a scheduled Gold table?
- Serverless vs. classic SQL warehouse: what do you give up and gain?
- How does max-clusters scaling absorb the 9 a.m. dashboard burst?
- Which system table shows who spent what on SQL compute?

Week 7: AI/BI Dashboards & Genie

Outcomes: build AI/BI dashboards on governed Gold data, configure Genie spaces with trusted instructions, curate datasets and measures for self-service, and publish dashboards on a schedule.

- **Monday:** AI/BI dashboards: datasets, visualizations, parameters, and cross-filters.

- **Tuesday:** AI/BI Genie: natural-language analytics, instructions, example queries, and trusted assets.
- **Wednesday:** Semantic curation: certified Gold tables, measures, lineage into dashboards, and scheduled refresh/sharing.
- **Thursday: Hands-on Project:** Build an executive AI/BI dashboard on `gold.fact_orders` with three parameterized pages, then create a Genie space over the same schema with instructions that correctly answer “revenue by state last quarter”; capture the generated SQL and fix one wrong answer with a certified measure.
- **Friday: Use Case & Architecture:** Design the self-service analytics layer for a retailer where business users ask Genie ad-hoc questions while finance requires certified, reconciled numbers only.

Resources: AI/BI dashboards documentation (<https://docs.databricks.com/dashboards/>); Genie documentation (<https://docs.databricks.com/genie/>). **Self-check:**

- Why must Genie spaces sit on Gold, never Bronze?
- What do Genie “instructions” change about generated SQL?
- Dashboard dataset vs. direct table reference: what is the trade-off?
- How do you certify a number so finance can trust the dashboard?
- What does a scheduled dashboard subscription need that ad-hoc viewing does not?

Week 8: Unity Catalog Foundations & Data Governance

Outcomes: explain the metastore→catalog→schema→table hierarchy, grant least-privilege access with GRANT, use volumes for file data, and audit access with system tables.

- **Monday:** Unity Catalog architecture: metastores, catalogs, schemas, tables, volumes, and the three-level namespace.
- **Tuesday:** Privilege model: GRANT/REVOKE, ownership, groups, and row/column-level security with dynamic views.
- **Wednesday:** Volumes vs. external locations, managed vs. external tables, audit logging, and data discovery.
- **Thursday: Hands-on Project:** Create a catalog with `bronze/silver/gold` schemas, register an external volume, grant a read-only analyst group Gold-only access, build a dynamic view masking customer PII columns, and prove the mask by querying as the restricted principal.
- **Friday: Use Case & Architecture:** Design governance for a healthcare lakehouse where analysts query Gold, engineers write Silver, only pipelines write Bronze, and every access must be auditable for HIPAA.

Resources: Unity Catalog documentation (<https://docs.databricks.com/data-governance/unity-catalog/>); Data Governance with Unity Catalog whitepaper (<https://www.databricks.com/resources>). **Self-check:**

- What does a metastore bind together, and how many per region?
- Managed vs. external table: who owns the lifecycle of the files?

- Why use a dynamic view for masking instead of a copied sanitized table?
- Which privilege lets a group read Gold but nothing beneath it?
- Where does the audit record of “who selected what” live?

CAPSTONE PROJECT - Milestone 2: Build a Governed Analytics Lakehouse. Extend the Month 1 medallion lakehouse with optimized storage (liquid clustering on fact tables), a serverless SQL warehouse serving an AI/BI dashboard plus a Genie space, and Unity Catalog governance: analyst/engineer groups, dynamic-view PII masking, and audit evidence from system tables.

Acceptance Criteria

- Benchmarking
- AI/BI dashboard
- Access control
- Audit evidence
- Cost estimation
- Security

Milestone 2 Capstone: Reference Solution Sketch

- **Architecture:** Gold Delta tables (liquid clustered) → serverless SQL warehouse → AI/BI dashboard + Genie space; Unity Catalog enforces group-based grants and a masking dynamic view; system tables provide the audit trail.
- **Steps:** ALTER TABLE ... CLUSTER BY on the fact table and re-benchmark; create warehouse with auto-stop 10 min; build dashboard datasets from certified views; create analyst/engineer groups and GRANTS; write v_customer_masked with CASE WHEN is_member('analysts'); validate with a test user.
- **Hardest part — the masking view:**

```
CREATE OR REPLACE VIEW gold.v_customer_masked AS
SELECT customer_id,
       CASE WHEN is_member('pii_readers') THEN email
            ELSE sha2(email, 256) END AS email,
       region, created_at
FROM gold.dim_customer;
```

- **Pitfalls:** granting at catalog level when Gold-only was required; Genie instructions contradicting the certified measure; benchmarking before clustering and drawing the wrong conclusion.

3 Milestone 3: Batch ETL & Orchestration (Month 3)

Objective: Engineer production batch pipelines with PySpark, Delta Live Tables, Auto Loader, and Databricks Workflows.

Learning outcomes: tune PySpark jobs, declare pipelines with DLT expectations, ingest incrementally with Auto Loader and CDC, and orchestrate multi-task jobs with Workflows.

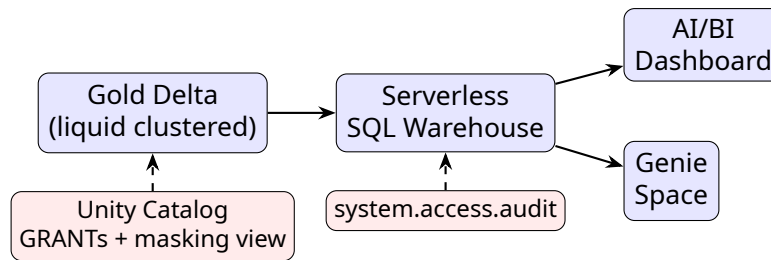


Figure 2: Milestone 2 reference architecture: governed, optimized analytics serving layer

Week 9: PySpark Deep Dive & Performance Tuning

Outcomes: choose between RDD, DataFrame, and SQL APIs deliberately, tune shuffle partitions and joins (broadcast vs. sort-merge), handle skew with salting, and profile with the Spark UI and Ganglia.

- **Monday:** Catalyst and Tungsten under the hood: whole-stage codegen, memory management, and when to drop to RDDs (rarely).
- **Tuesday:** Joins deep dive: broadcast hash vs. sort-merge, AQE (adaptive query execution), and skew hints.
- **Wednesday:** Shuffle tuning: `spark.sql.shuffle.partitions`, salting, caching strategy, and UDF performance (pandas UDFs).
- **Thursday: Hands-on Project:** Take the Week 4 job and tune it end-to-end: fix shuffle partition count, convert one join to broadcast, salt a skewed key, replace a Python UDF with a pandas UDF, and produce a before/after evidence table from Spark UI metrics.
- **Friday: Use Case & Architecture:** A partner's daily 2 TB join job misses its SLA by 3 hours: write the performance investigation report you would deliver, with the exact metrics and config changes.

Resources: Spark SQL performance tuning guide (<https://spark.apache.org/docs/latest/sql-performance-tuning.html>); The Definitive Guide to Apache Spark (Chambers & Zaharia).

Self-check:

- What does AQE change at runtime that static planning cannot?
- When does a broadcast join become dangerous?
- Why is a Python UDF slower than a pandas UDF?
- How does salting fix skew, and what does it cost the other side of the join?
- Which Spark UI tab tells you a stage is shuffle-bound?

Week 10: Delta Live Tables (Declarative Pipelines)

Outcomes: define streaming tables and materialized views declaratively, enforce quality with expectations (`expect`, `expect_or_drop`, `expect_or_fail`), run triggered vs. continuous pipelines, and read the DLT event log.

- **Monday:** DLT concepts: datasets, dependencies, declarative DAGs, and development vs. production modes.
- **Tuesday:** Expectations and data quality: warn/drop/fail semantics and quality metrics.

- **Wednesday:** Pipeline operations: triggered vs. continuous, full refresh, schema evolution handling, and the event log.
- **Thursday: Hands-on Project:** Rebuild the Month 1 Bronze→Silver→Gold flow as a DLT pipeline with expectations dropping bad `order_ids` and failing on null `customer_ids`; inject a poison record, observe drop metrics, and query the event log for quality statistics.
- **Friday: Use Case & Architecture:** Choose DLT vs. hand-written Spark + Workflows for three real pipeline candidates at a logistics company, and defend the boundary you draw.

Resources: Delta Live Tables documentation (<https://docs.databricks.com/delta-live-tables/>); DLT expectations guide (<https://docs.databricks.com/delta-live-tables/expectations.html>).

Self-check:

- What does the DLT DAG give you that a notebook chain does not?
- `expect_or_drop` vs. `expect_or_fail`: pick one for a PII column and justify.
- Why is the event log itself a Delta table worth querying?
- When is a full refresh required even in an incremental pipeline?
- Continuous vs. triggered: which suits a nightly cost-sensitive load?

Week 11: Auto Loader, Incremental Ingestion & CDC

Outcomes: ingest files incrementally with Auto Loader (`cloudFiles`) in directory and notification modes, rescue malformed records with `rescuedDataColumn`, apply CDC with `MERGE` and `apply_changes` patterns, and manage schema inference/evolution.

- **Monday:** Auto Loader architecture: `cloudFiles` source, directory listing vs. file notification mode, checkpoints, and exactly-once semantics.
- **Tuesday:** Schema inference, schema hints, evolution, and the rescued data column.
- **Wednesday:** CDC patterns: `MERGE INTO` upserts, SCD Type 1/2 with `AUTO CDC ... INTO` (formerly `apply_changes`), and delete handling.
- **Thursday: Hands-on Project:** Drop daily CSV batches into a UC volume, ingest them with an Auto Loader streaming query in triggered mode, simulate a schema change mid-stream plus a corrupted row, then apply a CDC feed to a Silver table with SCD Type 2 history for customers.
- **Friday: Use Case & Architecture:** Design ingestion for a payments platform receiving 500 small files/minute from partners plus a nightly CDC extract from the core banking database.

Resources: Auto Loader documentation (<https://docs.databricks.com/ingestion/auto-loader/>); SCD with DLT documentation (<https://docs.databricks.com/delta-live-tables/cdc.html>). **Self-check:**

- Notification vs. directory-listing mode: when does the choice matter at scale?
- What makes Auto Loader exactly-once across restarts?
- What lands in `_rescued_data` and why is that better than failing?
- SCD Type 1 vs. Type 2: what does each answer about history?

- How does SEQUENCE BY resolve out-of-order CDC events?

Production Note: Auto Loader's checkpoint is the pipeline's memory: never delete it to "fix" a stuck stream — you will reprocess or skip files. If you must reset, version the target table first or plan a full backfill with reconciliation.

Week 12: Workflows, Jobs & Pipeline Orchestration

Outcomes: build multi-task jobs with dependencies, parameterize tasks, configure retries and alerts, orchestrate DLT pipelines and notebooks together, and use `dbutils.notebook.run` vs. task values correctly.

- **Monday:** Jobs and tasks: DAGs of tasks, job clusters vs. shared clusters, and scheduling with cron.
- **Tuesday:** Parameters, task values, conditionals (`if/else` tasks), and for-each tasks.
- **Wednesday:** Reliability: retries, timeouts, alerts, repair runs, and monitoring with system tables.
- **Thursday: Hands-on Project:** Orchestrate the full medallion flow as one job: Auto Loader ingestion task → DLT pipeline task → data-quality notebook → conditional alert task on failure, parameterized by `processing_date`; force a failure and execute a repair run of only the failed tasks.
- **Friday: Use Case & Architecture:** Design the orchestration layer for 60 interdependent nightly pipelines across three teams with a shared 6 a.m. SLA, including backfill and failure-isolation strategy.

Resources: Databricks Workflows documentation (<https://docs.databricks.com/workflows/>); Jobs API reference (<https://docs.databricks.com/api/workspace/jobs>). **Self-check:**

- Why give each task its own job cluster in a critical pipeline?
- Task values vs. job parameters: which is computed at runtime?
- What does a repair run re-execute, and why is that cheaper than a full rerun?
- Where should a data-quality gate sit relative to Gold publishing?
- How do `if/else` tasks replace external alerting glue?

CAPSTONE PROJECT - Milestone 3: Build an Orchestrated Batch ELT Platform. Ingest the full Olist dataset daily via Auto Loader into Bronze, transform with a DLT pipeline enforcing expectations into Silver/Gold, orchestrate with a multi-task Workflow (ingest → pipeline → quality gate → publish) parameterized by date, with retries, failure alerts, and a demonstrated repair run.

Acceptance Criteria

- One W (rerun)
- DLT ex (dropped)
- Auto L (without)
- Pipelin
- Cost es (mented)
- Securit (location)

Milestone 3 Capstone: Reference Solution Sketch

- **Architecture:** partner files → Auto Loader (volume + checkpoint) → Bronze streaming table → DLT Silver with expectations → Gold publish task → conditional failure alert.
- **Steps:** create the volume and checkpoint schema; author the DLT pipeline; build the Workflow with task values passing `processing_date`; add the quality-gate notebook asserting reconciliation; wire the if/else alert task; run, break, repair.
- **Hardest part — the quality gate as a task:**

```
bad = spark.sql("""
    SELECT COUNT(*) c FROM silver.orders
    WHERE order_id IS NULL OR total_amount < 0
""").first().c
if bad > 0:
    raise ValueError(f"Quality gate failed: {bad} bad rows")
dbutils.jobs.taskValues.set("quality", "passed")
```

- **Pitfalls:** deleting the Auto Loader checkpoint and double-ingesting; expectations logging but not wired to `expect_or_fail` where it matters; a shared cluster for all tasks turning one OOM into six failed tasks.

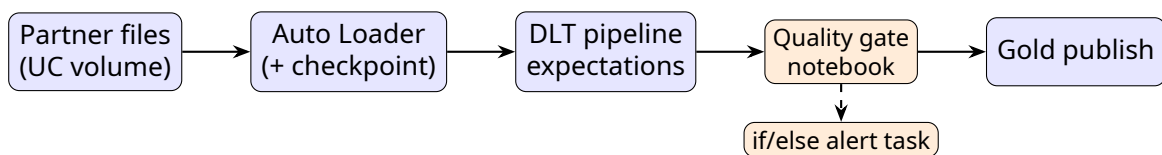


Figure 3: Milestone 3 reference architecture: orchestrated, quality-gated batch ELT

4 Milestone 4: Streaming & Real-Time (Month 4)

Objective: Build production streaming pipelines with Structured Streaming, stateful processing, and Kafka integration.

Learning outcomes: reason in event time with windows and watermarks, guarantee exactly-once with checkpoints and idempotent sinks, integrate Kafka/Event Hubs/Kinesis, and architect real-time lakehouse systems.

Week 13: Structured Streaming Foundations

Outcomes: explain the micro-batch execution model, read from streaming sources, write to Delta with checkpoints and triggers, and monitor a stream with `StreamingQueryListener` and the UI.

- **Monday:** Structured Streaming model: the infinite table, micro-batches vs. continuous, and the Catalyst integration.
- **Tuesday:** Sources, sinks, output modes (append/update/complete), and triggers including `availableNow`.
- **Wednesday:** Checkpoints, offsets, recovery semantics, and stream monitoring.

- **Thursday: Hands-on Project:** Build a streaming pipeline from a simulated clickstream (rate source or files dropped into a volume) into a Delta sink; run with three different triggers, kill and restart the query mid-stream, and prove from the checkpoint that no events were lost or duplicated.
- **Friday: Use Case & Architecture:** Design the streaming ingestion layer for an ad-tech platform receiving 50k events/second that must land queryable data within one minute.

Resources: Structured Streaming programming guide (<https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>); Databricks streaming documentation (<https://docs.databricks.com/structured-streaming/>). **Self-check:**

- Why is a checkpoint directory mandatory for a production stream?
- Append vs. update vs. complete output mode: which supports aggregation to Delta?
- What does `availableNow` trigger give a streaming job?
- Why can't you change a streaming aggregation's schema freely on restart?
- Which metric tells you the stream is falling behind?

Production Note: *One checkpoint location, one stream definition: pointing a second query at the same checkpoint corrupts offsets. Treat checkpoint paths as part of the pipeline contract — version them in your repo README and govern them in Unity Catalog volumes.*

Week 14: State Management, Watermarks & Exactly-Once

Outcomes: implement event-time windows with watermarks, manage stateful aggregations and deduplication (`dropDuplicates`, `withWatermark`), use `flatMapGroupsWithState` and the RocksDB state store, and reason about exactly-once end-to-end.

- **Monday:** Event time vs. processing time; tumbling, sliding, and session windows.
- **Tuesday:** Watermarks: late-data thresholds, state eviction, and their interaction with output modes.
- **Wednesday:** Stateful operators: deduplication, arbitrary stateful processing, state store backends (HDFS vs. RocksDB), and exactly-once composition with idempotent sinks.
- **Thursday: Hands-on Project:** Build a windowed aggregation over out-of-order IoT events with a 10-minute watermark, inject late and duplicate events, prove dedup via `dropDuplicates` on `event_id`, and compare state-store memory with and without RocksDB.
- **Friday: Use Case & Architecture:** Design a fraud-detection stream that must tolerate 15 minutes of late events, deduplicate replays, and still emit alerts within 30 seconds of event time.

Resources: Structured Streaming watermarking docs (<https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html#handling-late-data-and-watermarking>); RocksDB state store guide (<https://docs.databricks.com/structured-streaming/rocksdb-state-store.html>). **Self-check:**

- What does a watermark actually delete from memory?
- Why does exactly-once require both a checkpoint *and* an idempotent sink?
- Tumbling vs. sliding window: what does the slide add?
- When must you switch from the default state store to RocksDB?
- Why does `dropDuplicates` without a watermark risk unbounded state?

Week 15: Kafka, Event Hubs & Kinesis Integration

Outcomes: connect Structured Streaming to Apache Kafka (and Kafka-compatible Event Hubs) and Amazon Kinesis, manage offsets and `startingOffsets/maxOffsetsPerTrigger`, deserialize JSON/Avro, and design consumer groups for scale.

- **Monday:** Kafka architecture for data engineers: topics, partitions, consumer groups, and offset semantics.
- **Tuesday:** Spark Kafka source: options, SSL/SASL authentication, schema registry and Avro, Event Hubs via the Kafka endpoint.
- **Wednesday:** Kinesis source, rate limiting with `maxOffsetsPerTrigger`, backpressure, and multi-stream topologies.
- **Thursday: Hands-on Project:** Run a local Kafka (Docker) producing synthetic transactions, consume with Structured Streaming into Bronze Delta, enforce a 1k-records-per-trigger cap, then replay from `startingOffsets="earliest"` into a recovery table and reconcile counts.
- **Friday: Use Case & Architecture:** Design the messaging backbone for a retailer migrating from on-prem Kafka to Event Hubs while keeping Databricks consumers unchanged and zero-downtime.

Resources: Spark Kafka integration guide (<https://spark.apache.org/docs/latest/structured-streaming-kafka-integration.html>); Event Hubs Kafka endpoint docs (<https://learn.microsoft.com/azure/event-hubs/event-hubs-for-kafka>)

Self-check:

- Why does Kafka give at-least-once delivery, and where do you add idempotence?
- What does `maxOffsetsPerTrigger` protect?
- Why prefer a schema registry over inferred JSON schemas for streaming?
- How do Kafka partitions map to Spark tasks?
- What is the Event Hubs trick that lets Kafka clients connect unchanged?

Week 16: Real-Time Lakehouse Architectures

Outcomes: design streaming medallion architectures, combine stream and batch in one Delta table, serve real-time dashboards from Gold streaming tables, and operate latency/cost trade-offs deliberately.

- **Monday:** Streaming medallion: Bronze append streams, Silver streaming joins and dedup, Gold incremental aggregates.
- **Tuesday:** Stream-static and stream-stream joins, CDC-to-stream fan-in, and `foreachBatch` for merge-based sinks.
- **Wednesday:** Serving real-time Gold to AI/BI dashboards, latency budgets, and cost models for continuous vs. triggered processing.
- **Thursday: Hands-on Project:** Build a real-time orders pipeline: Kafka → Bronze stream → Silver deduped stream with `foreachBatch MERGE` → Gold 5-minute tumbling revenue table, visualized on an AI/BI dashboard refreshing every minute.

- **Friday: Use Case & Architecture:** Architect a real-time lakehouse for a logistics fleet: 10k vehicles × 1 event/second, sub-minute dispatch dashboards, and a daily batch reconciliation that must agree with the stream.

Resources: Databricks real-time architecture patterns (<https://www.databricks.com/blog>); Structured Streaming + Delta guide (<https://docs.databricks.com/structured-streaming/delta-lake.html>). **Self-check:**

- Why is `foreachBatch` the escape hatch for MERGE sinks?
- How can one Delta table serve both stream and batch writers safely?
- What latency budget forces continuous processing over 1-minute triggers?
- Why keep a batch reconciliation path even in a “real-time” system?
- What does a streaming Gold table cost per day vs. a triggered one?

CAPSTONE PROJECT - Milestone 4: Build a Real-Time Fraud Telemetry Platform. Stream synthetic card transactions from Kafka into a Bronze Delta table, deduplicate and window in Silver with a watermark, flag impossible-travel fraud with a stateful rule, publish Gold 5-minute aggregates to a live AI/BI dashboard, and prove exactly-once via a replay drill with zero duplicate alerts.

Acceptance Criteria

- Replay duplicates
- Watermark
- Impossible-travel
- Dashboard aggregator
- Cost efficiency
- Security
- Bronze table

Milestone 4 Capstone: Reference Solution Sketch

- **Architecture:** Kafka → Bronze (append stream) → Silver (dropDuplicates + watermark + `foreachBatch` MERGE) → Gold tumbling aggregates → AI/BI dashboard; the stateful impossible-travel rule runs as a second Silver stream.
- **Steps:** stand up the Kafka producer with unique `event_ids`; configure checkpoints per query; set `maxOffsetsPerTrigger`; implement dedup before rules; build the dashboard; run the replay drill.
- **Hardest part — dedup-before-rules ordering:**

```
silver = (bronze_stream
    .withWatermark("event_ts", "10 minutes")
    .dropDuplicates(["event_id"])
    .groupBy(window("event_ts", "5 minutes"), "card_id")
    .agg(count("*").alias("txns"),
        countDistinct("city").alias("cities")))

```

- **Pitfalls:** rules reading the raw (undeduplicated) stream double-fire on replay; watermark shorter than real lateness silently drops valid events; a continuous trigger left running over the weekend burns the week’s budget.

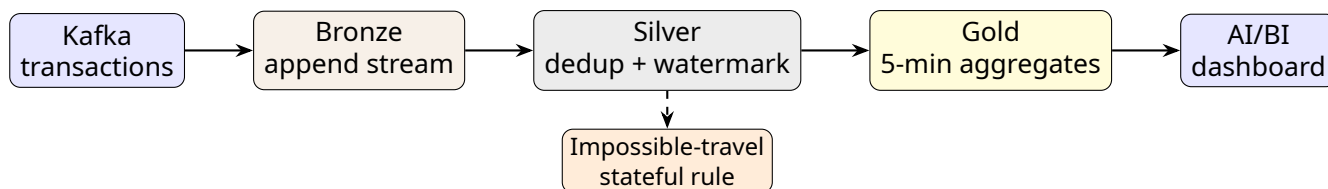


Figure 4: Milestone 4 reference architecture: real-time fraud telemetry lakehouse

5 Milestone 5: ML & MLOps (Month 5)

Objective: Productionize machine learning on Databricks with MLflow, the Feature Store, Mosaic AI, and model serving.

Learning outcomes: track experiments and register models with MLflow, build leakage-safe feature tables, assemble a RAG pipeline with Vector Search, and serve and monitor models with governance.

Week 17: MLflow & Experiment Tracking

Outcomes: track parameters, metrics, and artifacts with MLflow autologging, compare runs in the UI, register models in Unity Catalog, and reproduce a training run from its logged environment.

- **Monday:** MLflow components: Tracking, Models, Registry (in Unity Catalog), and the run/artifact model.
- **Tuesday:** Autologging, manual logging, run tagging with dataset version and git commit, and model signatures.
- **Wednesday:** Model registry workflows: aliases (champion/challenger), versioning, approval, and loading models by alias.
- **Thursday: Hands-on Project:** Train six scikit-learn churn models with MLflow autologging, tag each run with dataset snapshot and commit hash, register the best as `champion`, and load it back in a fresh notebook via `models:/catalog.schema.churn_model@champion`.
- **Friday: Use Case & Architecture:** Design the experiment-governance process for a bank where every production model must trace to data, code, hyperparameters, and an approver.

Resources: MLflow documentation (<https://mlflow.org/docs/latest/>); MLflow on Databricks guide (<https://docs.databricks.com/mlflow/>). **Self-check:**

- What does autologging capture that manual logging usually forgets?
- Why register models in Unity Catalog instead of the workspace registry?
- Alias vs. version number: which should production jobs reference?
- What makes a run reproducible a year later?
- Why compare runs only on identical data snapshots?

Week 18: Feature Engineering & Feature Store

Outcomes: build feature tables with `FeatureEngineeringClient`, enforce point-in-time correctness with `create_training_set`, serve online features, and prevent training-serving skew.

- **Monday:** Feature engineering discipline: leakage, point-in-time correctness, and feature definitions as code.
- **Tuesday:** Feature Store in Unity Catalog: feature tables, `create_training_set` with timestamp lookup, and lineage to models.
- **Wednesday:** Online stores and feature serving, on-demand features, and feature freshness monitoring.
- **Thursday: Hands-on Project:** Build `customer_features` and `order_features` tables with as-of timestamps, create a training set with point-in-time joins, train a model that records feature-table lineage, and verify the same features are served at inference.
- **Friday: Use Case & Architecture:** Audit a churn model that performs 15 points better in training than production: find the leakage and redesign the feature pipeline.

Resources: Feature Engineering in Unity Catalog docs (<https://docs.databricks.com/machine-learning/feature-store/>); point-in-time join guide (<https://docs.databricks.com/machine-learning/feature-store/time-series.html>). **Self-check:**

- What is label leakage and give one concrete example?
- How does a timestamp lookup prevent future data entering training rows?
- Why does the registry record feature-table lineage?
- Online vs. offline feature store: which serves inference at 10 ms?
- What is training-serving skew and how does the Feature Store prevent it?

Week 19: Mosaic AI, Vector Search & GenAI (RAG)

Outcomes: build embeddings pipelines, create and query Mosaic AI Vector Search indexes, assemble a retrieval-augmented generation (RAG) chain, and evaluate answer quality with MLflow evaluation.

- **Monday:** GenAI building blocks on Databricks: Foundation Model APIs, embeddings, and Model Serving endpoints.
- **Tuesday:** Vector Search: managed vs. Delta-sync indexes, embedding models, similarity search, and metadata filters.
- **Wednesday:** RAG architecture: chunking, retrieval, prompt assembly, evaluation with MLflow, and governance of GenAI pipelines.
- **Thursday: Hands-on Project:** Build a RAG assistant over this program's own documentation: chunk markdown into a Delta table, sync a Vector Search index, retrieve top-k context for a question, and assemble an answer via a Foundation Model endpoint; evaluate 10 questions with MLflow and log the quality metrics.
- **Friday: Use Case & Architecture:** Design a customer-support copilot over 2M help-center articles with freshness requirements, PII redaction, and a cost ceiling per answer.

Resources: Mosaic AI documentation (<https://docs.databricks.com/machine-learning/>); Vector Search documentation (<https://docs.databricks.com/generative-ai/vector-search.html>).

Self-check:

- Why store source chunks in Delta and sync the index instead of writing vectors directly?
- Managed vs. Delta-sync index: which do you choose for a table that updates hourly?
- What does chunk size trade off in retrieval quality?
- Why evaluate RAG with logged metrics rather than eyeballing answers?
- Where should PII redaction happen in the RAG pipeline?

Week 20: Model Serving, Monitoring & MLOps

Outcomes: deploy models to serverless serving endpoints, implement batch scoring with idempotent writes, monitor input drift and prediction quality with Lakehouse Monitoring, and gate promotion behind metric thresholds.

- **Monday:** Model Serving: serverless endpoints, scaling to zero, provisioned concurrency, and endpoint governance.
- **Tuesday:** Batch and streaming scoring patterns: alias-loaded models as Spark UDFs, partition-idempotent writes.
- **Wednesday:** Lakehouse Monitoring: inference tables, drift metrics, alerting, and the re-training loop with human-gated promotion.
- **Thursday: Hands-on Project:** Deploy the Week 17 champion to a serverless endpoint, run a batch-scoring job over 100k rows with `replaceWhere` idempotency, enable an inference table, inject skewed inputs, and prove a monitor catches the drift while a retraining trigger fires but does not auto-promote.
- **Friday: Use Case & Architecture:** Design the MLOps control plane for a lender whose regulator requires drift evidence, approval records, and rollback for every model in production.

Resources: Model Serving documentation (<https://docs.databricks.com/machine-learning/model-serving/>); Lakehouse Monitoring documentation (<https://docs.databricks.com/lakehouse-monitoring/>).

Self-check:

- What does an inference table capture and why does that enable monitoring?
- Why load models by alias in scoring jobs?
- What makes a batch-scoring write idempotent?
- Why must retraining triggers not auto-promote to production?
- Scale-to-zero vs. provisioned concurrency: which fits a latency-critical endpoint?

Acceptance

- Training review
- All runs book.
- RAG pi
- Batch s record
- Drift m human
- Security docum

CAPSTONE PROJECT - Milestone 5: Build an End-to-End MLOps Pipeline. Engineer leakage-safe features into Unity Catalog feature tables, train and track churn models with MLflow, register a champion, build a RAG knowledge assistant over the platform's runbooks with Vector Search, serve the model on a serverless endpoint, and operate drift monitoring with a human-gated retraining loop.

Milestone 5 Capstone: Reference Solution Sketch

- **Architecture:** Gold → feature tables (as-of timestamps) → MLflow-tracked training → UC registry (champion alias) → serverless endpoint + inference table → Lakehouse monitor → gated retraining; RAG branch: docs → chunks Delta → Vector Search → Foundation Model endpoint.
- **Steps:** author feature jobs with `as_of_date` parameters; build training set with timestamp lookups; autolog six runs; promote by alias; deploy endpoint; enable monitoring; inject drift; observe gate.
- **Hardest part — point-in-time training set:**

```
training_set = fs.create_training_set(  
    df=labels_df,                # has customer_id + event_ts  
    feature_lookups=[  
        FeatureLookup("ml.customer_features",  
                        lookup_key="customer_id",  
                        timestamp_lookup_key="event_ts")],  
    label="churned",  
    exclude_columns=["customer_id", "event_ts"])
```

- **Pitfalls:** features computed past their as-of date (leakage by construction); scoring against ad hoc features instead of the governed table; an unmonitored endpoint whose drift you discover from a business stakeholder.

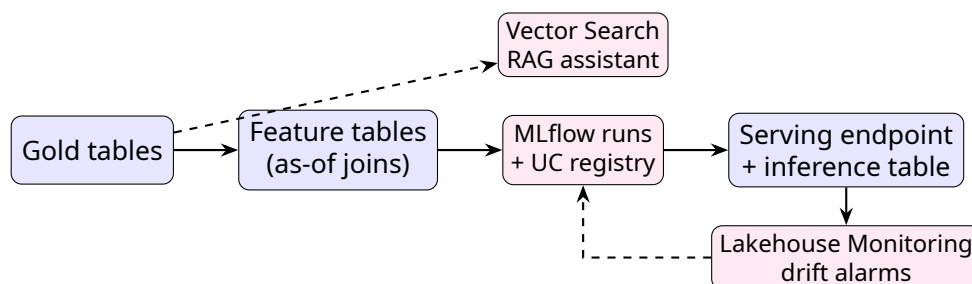


Figure 5: Milestone 5 reference architecture: governed MLOps with monitoring and RAG

6 Milestone 6: Security, Production & Certification (Month 6)

Objective: Harden the platform, operate deep governance, automate delivery with Asset Bundles and Terraform, and prepare for certification.

Learning outcomes: implement identity and network isolation, exploit Unity Catalog lineage/audit/sharing, deploy with Databricks Asset Bundles and Terraform, optimize cost, and pass the Databricks Certified Data Engineer exam.

Week 21: Security, IAM, SCIM & Network Isolation

Outcomes: design account/workspace identity with SSO and SCIM provisioning, use service principals and secret scopes, configure network isolation (Private Link/PrivateLink-style connectivity, IP access lists), and apply least privilege across compute and data.

- **Monday:** Identity architecture: account console vs. workspace, SSO with the IdP, SCIM provisioning, and group strategy.
- **Tuesday:** Service principals, OAuth machine-to-machine, personal access token policy, and secret scopes (Databricks-backed vs. Key Vault-backed).
- **Wednesday:** Network security: private connectivity, IP access lists, storage firewall rules, and audit-ready network design.
- **Thursday: Hands-on Project:** Provision a service principal via the CLI, grant it only the schemas it needs, store its credentials in a secret scope, convert a notebook from a hard-coded PAT to `dbutils.secrets.get`, and add an IP access list; verify a blocked connection is denied.
- **Friday: Use Case & Architecture:** Design the security architecture for a multi-team platform where contractors get 90-day access, production jobs run as service principals, and all traffic must stay off the public internet.

Resources: Databricks security documentation (<https://docs.databricks.com/security/>); SCIM provisioning guide (<https://docs.databricks.com/admin/users-groups/scim/>). **Self-check:**

- Why do production jobs run as service principals, not people?
- What does SCIM automate that manual user management cannot?
- Databricks-backed vs. Key Vault-backed secret scope: who controls rotation?
- What does an IP access list *not* protect against?
- How does private connectivity change your threat model?

Week 22: Unity Catalog Deep Dive: Lineage, Audit & Delta Sharing

Outcomes: read column-level lineage from system tables, build an audit practice on `system.access.audit`, share data securely with Delta Sharing (open protocol and Databricks-to-Databricks), and implement data classification and tag-based policies.

- **Monday:** Lineage: table and column lineage, capturing it, and using it for impact analysis and debugging.
- **Tuesday:** Audit deep dive: `system.access.audit` schema, alerting on sensitive access, and compliance reporting.

- **Wednesday:** Delta Sharing: shares, recipients, clean rooms, and open-protocol vs. Databricks-to-Databricks sharing.
- **Thursday: Hands-on Project:** Build a lineage report tracing `gold.fact_orders.total_amount` back to Bronze columns, write an audit alert for any read of the PII-mapped columns by a non-approved group, and share a Gold table to an external recipient via Delta Sharing with a documented revocation test.
- **Friday: Use Case & Architecture:** Design data sharing for a supply-chain consortium: five companies exchange curated datasets with auditability, revocation, and no cross-company credentials.

Resources: Unity Catalog lineage documentation (<https://docs.databricks.com/data-governance/unity-catalog/data-lineage.html>); Delta Sharing documentation (<https://docs.databricks.com/delta-sharing/>). **Self-check:**

- How does column-level lineage accelerate a breaking-change impact analysis?
- Which system table records “who ran what query”?
- Why does Delta Sharing avoid giving consumers your storage credentials?
- What happens to a recipient’s access when you revoke a share?
- Why is open-protocol sharing important for non-Databricks consumers?

Week 23: CI/CD with Databricks Asset Bundles, Terraform & Cost Optimization

Outcomes: define jobs/pipelines as code with Databricks Asset Bundles (YAML), provision infrastructure with the Terraform Databricks provider, wire GitHub Actions CI/CD with dev/staging/prod targets, and build a cost-optimization practice from system billing tables.

- **Monday:** Databricks Asset Bundles: `databricks.yml`, resources, targets, variables, and `bundle deploy/validate/run`.
- **Tuesday:** Terraform provider for Databricks: workspace resources, drift detection, and state management.
- **Wednesday:** CI/CD pipelines: GitHub Actions with OIDC/service-principal auth, PR validation, and dev→staging→prod promotion; cost analysis with `system.billing.usage`.
- **Thursday: Hands-on Project:** Convert the Month 3 Workflow into an Asset Bundle with dev and prod targets, validate and deploy from GitHub Actions on merge, provision the target workspace resources with Terraform, and produce a weekly DBU cost report with two implemented optimizations.
- **Friday: Use Case & Architecture:** Design the delivery and FinOps operating model for a platform team shipping 30 pipelines across three environments with monthly budget accountability per domain.

Resources: Databricks Asset Bundles documentation (<https://docs.databricks.com/dev-tools/bundles/>); Terraform Databricks provider (<https://registry.terraform.io/providers/databricks/databricks/latest/docs>). **Self-check:**

- What does `databricks bundle validate` catch before deployment?
- Why separate bundle targets per environment rather than parameter flags?

- Terraform state: why must it be remote and locked?
- Which system table powers chargeback reports?
- Name two compute settings that cut DBU spend without slowing jobs.

Production Note: *Pin the deployed branch, not a floating one: a bundle that deploys `main` on every merge without a staging gate reproduces the classic “Friday 5 p.m. broken prod pipeline.” Require the staging target to run green in CI before prod deployment is permitted.*

Week 24: Final Capstone & Databricks Data Engineer Certification Prep

Outcomes: integrate all six milestones into one production-grade platform, execute a certification study plan for the Databricks Certified Data Engineer Associate/Professional, and complete two timed mock exams.

- **Monday:** Full-platform review: lakehouse, warehousing, ETL, streaming, ML, and governance — one integrated architecture map.
- **Tuesday:** Certification domains walkthrough: Databricks platform, Delta Lake, ETL with Spark/DLT, production pipelines, and governance.
- **Wednesday:** Exam drills: scenario questions, time management, and weak-domain remediation by rebuilding labs.
- **Thursday: Hands-on Project:** Final Capstone build (see below): the complete governed data platform deployed from an Asset Bundle with CI/CD, monitoring, cost report, and security review.
- **Friday: Use Case & Architecture:** Present the final capstone as a 15-minute architecture review defending three design decisions with trade-offs, one failure story, and cost numbers.

Resources: Databricks certification page (<https://www.databricks.com/learn/certification>); Databricks Academy self-paced courses (<https://www.databricks.com/learn/training>). **Self-check:**

- Which exam domain is your weakest, and which lab rebuilds it?
- Can you deploy your whole platform from a clean clone? Prove it.
- What three numbers (cost, latency, rows) summarize your platform?
- Which single design decision would you revisit at 10x scale?
- Can you explain every box on your architecture diagram in one sentence?

Acceptance

- Clean-
builds
- Batch a
duplica
- Govern
share v
- CI/CD:
- Cost re
menter
- Two co

CAPSTONE PROJECT - Milestone 6: Final Capstone: The Governed Data Platform. Deploy a complete platform — medallion lakehouse with optimized Delta storage, DLT pipelines with expectations, Auto Loader + Kafka streaming, MLflow-managed model with serving and monitoring, Unity Catalog governance with lineage/audit/Delta Sharing — as code via Asset Bundles and Terraform, promoted through dev/staging/prod by GitHub Actions, with a cost report and a certification-style architecture defense.

Milestone 6 Capstone: Reference Solution Sketch

- **Architecture:** one bundle defining volumes, pipelines, jobs, serving endpoint, and monitors across three targets; Terraform provisioning workspace-level resources; Unity Catalog enforcing the access matrix; system tables powering audit and cost reporting.
- **Steps:** extract every hand-built artifact from Months 1–5 into bundle YAML; define dev/staging/prod targets with variables; wire GitHub Actions with service-principal auth; run the redeploy drill in a fresh workspace; present.
- **Hardest part — the bundle skeleton:**

```
bundle:
  name: governed-platform
targets:
  prod:
    mode: production
    workspace:
      host: https://<workspace>.azuredatabricks.net
resources:
  jobs:
    nightly_elt:
      name: nightly-elt
      tasks:
        - task_key: ingest
          pipeline_task: {pipeline_id: \${resources.pipelines.medallion.id}}
```

- **Pitfalls:** hardcoded workspace IDs breaking the prod target; Terraform state in a local file; a demo that works only because your personal credentials are cached — test with the service principal.

7 Appendix A: Glossary

Delta Lake	Open table format adding ACID transactions, time travel, and schema enforcement to Parquet files in cloud storage.
Transaction Log	The <code>_delta_log</code> directory of JSON commits (plus checkpoints) making Delta writes atomic and versions queryable.

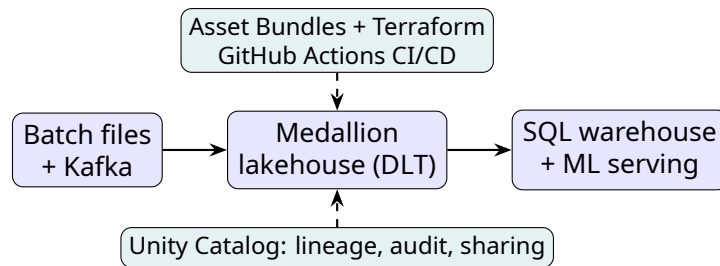


Figure 6: Milestone 6 reference architecture: the complete governed, code-deployed platform

Medallion Architecture

Progressive-refinement lakehouse pattern: Bronze (raw), Silver (validated), Gold (curated).

Unity Catalog	Databricks' unified governance layer: metastore → catalog → schema → table/volume, with lineage and audit.
Volume	Unity Catalog object governing access to files in cloud storage for non-tabular data.
DBU	Databricks Unit — the compute billing unit consumed per second of cluster or serverless runtime.
Photon	Databricks' vectorized C++ query engine accelerating SQL and DataFrame workloads.
SQL Warehouse	SQL-optimized compute (serverless, pro, or classic) powering Databricks SQL, dashboards, and Genie.
AI/BI Dashboard	Databricks-native BI artifact with parameterized datasets and visualizations on governed data.
Genie	AI/BI natural-language interface that generates SQL over curated, instructed datasets.
OPTIMIZE	Delta command compacting small files, optionally with <code>ZORDER BY</code> multi-dimensional clustering.
Liquid Clustering	Delta layout technique (<code>CLUSTER BY</code>) replacing fixed partitions/ <code>Z-ORDER</code> with rewritable-free clustering keys.
Deletion Vector	Delta row-level delete marker avoiding full file rewrites for <code>DELETE/UPDATE/MERGE</code> .
Auto Loader	Incremental file-ingestion source (<code>cloudFiles</code>) with checkpointed exactly-once semantics and schema evolution.
DLT (Delta Live Tables)	Declarative pipeline framework with streaming tables, materialized views, and data-quality expectations.
Expectation	DLT data-quality rule with warn (<code>expect</code>), drop, or fail semantics.
Checkpoint	Structured Streaming (or Auto Loader) state directory enabling exactly-once recovery after restarts.
Watermark	Event-time threshold bounding how late events are accepted and when state is evicted.
Task Value	Runtime value passed between tasks in a Databricks Workflow job.
MLflow	Open-source ML lifecycle platform: tracking, models, registry (in Unity Catalog on Databricks), and evaluation.

Feature Store	Governed feature tables with point-in-time-correct training sets and serving lineage.
Mosaic AI Vector Search	Managed vector index over Delta tables powering semantic retrieval for RAG.
Model Serving	Serverless (or provisioned) endpoints exposing registered models, with inference tables for monitoring.
Lakehouse Monitoring	Managed drift and quality monitoring over Delta tables and model inference logs.
Delta Sharing	Open protocol for sharing live Delta tables to any recipient without copying data or sharing storage credentials.
Asset Bundle	Databricks' infrastructure-as-code format (<code>databricks.yml</code>) defining jobs, pipelines, and targets for CI/CD.
Service Principal	Non-human identity for automated jobs and CI/CD, granted least-privilege access.
Secret Scope	Governed vault reference (Databricks- or Key Vault-backed) injected via <code>dbutils.secrets</code>

8 Appendix B: Assessment & Grading Model

Each monthly capstone is graded on the following weighted rubric:

Category	Weight	What is evaluated
Correctness	30%	Pipeline runs end-to-end and produces the specified outputs from a clean environment.
Reliability & tests	20%	Data-quality expectations, failure handling, retries/repair runs, and a demonstrated failing case.
Security	15%	Least-privilege service principals, secret scopes, no hardcoded tokens, PII masking.
Cost awareness	15%	DBU-based monthly estimate with at least 2 documented optimization decisions.
Documentation	10%	README, architecture diagram, and run-book quality.
Demo	10%	Live or recorded walkthrough of the working system.

The two full-length mock exams scheduled for the weekend after Week 24 feed the final certification readiness score: a consistent score of 80%+ across both mocks indicates readiness to book the Databricks Certified Data Engineer exam; below that, revisit the weak domains noted during Week 24 Friday before booking.

9 Appendix C: Interview Preparation

1. Explain the Delta transaction log and why it makes a data lake ACID-compliant. (Week 2)
2. When does liquid clustering beat partition + Z-ORDER, and when would you keep partitioning? (Week 5)
3. A Databricks SQL dashboard burst at 9 a.m. queues for minutes — how do you fix it? (Week 6)
4. Design Unity Catalog grants so analysts read masked Gold while engineers write Silver. (Week 8)
5. A PySpark join is skewed on one key — walk through your Spark UI evidence and the salting fix. (Week 9)
6. DLT vs. plain Spark + Workflows: where do you draw the line, and how do expectations change your testing? (Weeks 10, 12)
7. How does Auto Loader guarantee exactly-once ingestion, and what breaks if someone deletes the checkpoint? (Week 11)
8. Design a streaming fraud pipeline with watermarks, dedup, and exactly-once semantics — end to end. (Weeks 13–14)
9. How do you prove a training set has no label leakage, and what does the Feature Store contribute? (Week 18)
10. Design a governed multi-environment delivery model with Asset Bundles, Terraform, and per-domain chargeback. (Weeks 22–23, Capstone 6)

10 Appendix D: Self-Check Answer Key

Terse answers to the weekly self-check questions.

Week 1:

1. Workspace metadata, notebooks, and the web UI live in the control plane; cluster VMs and your data live in the data plane.
2. Job clusters start and stop per run and are priced lower; all-purpose clusters bill for interactive uptime.
3. It caps configuration choices (size, auto-termination, spot use), protecting budgets and standards.
4. A Git folder syncs with a remote Git branch; a plain workspace folder is not version-controlled.
5. OAuth issues short-lived tokens; a PAT is a long-lived secret that can leak from scripts.

Week 2:

1. The commit to `_delta_log` is a single atomic file write listing all data files for the version.
2. Optimistic concurrency with snapshot isolation; conflicting concurrent writes retry or fail.
3. `mergeSchema` for additive evolution; `overwriteSchema` for deliberate breaking rewrites.
4. The data files old versions reference; 7 days protects in-flight recovery and time travel.
5. MERGE is atomic and avoids the delete-then-insert window where readers see missing rows.

Week 3:

1. Bronze is the replayable evidence layer — every downstream fix starts from raw.
2. Rows that fail validation, with the rejection reason, so nothing is silently lost.
3. Partition-scoped `replaceWhere` and key-based MERGE make reruns converge to the same state.
4. A star schema gives stable, well-tested dimensions and selective fact scans; wide tables recompute everything.
5. `_ingested_at`, `_source_file`, and pipeline run IDs.

Week 4:

1. The driver plans once against the full DAG, enabling predicate pushdown and join reordering.
2. Wide transformations (`groupBy`, `join`) move data across executors — network and disk cost.
3. `repartition` shuffles and can increase or decrease; `coalesce` only decreases without a full shuffle.
4. Partition skew — one task processing far more records than its peers.
5. When cached data is scanned once, is huge, or forces memory pressure and eviction.

Week 5:

1. Every file costs listing, footer reads, and a task slot; metadata dominates at small-file scale.
2. High-cardinality columns used in selective equality/range filters.
3. The clustering columns — `ALTER TABLE ... CLUSTER BY` without rewriting layout decisions into paths.
4. Rewriting entire Parquet files to remove a handful of deleted rows.
5. Compaction rewrites files the stream may still be appending; coordinate schedules.

Week 6:

1. Vectorized execution of scans, joins, and aggregations — the difference between minutes and seconds on large scans.
2. When the result is queried repeatedly and source freshness tolerances allow scheduled refresh.
3. You give up warm-start control and gain instant start, fast autoscaling, and no cluster management.
4. The warehouse adds clusters up to the max during the burst, then scales back down.
5. `system.billing.usage`, joined to `system.access.audit` or warehouse events.

Week 7:

1. Genie reasons over semantics; Bronze's raw, uncleaned data produces confident wrong answers.
2. They steer table/column choice, join paths, and metric definitions for generated SQL.
3. A curated dataset documents semantics and reuse; direct references couple the dashboard to physical tables.
4. Certify the Gold view, reconcile it to the source, and lock editing with ownership + lineage.
5. A warehouse that can run unattended with governed credentials and a refresh schedule.

Week 8:

1. Metastores bind catalogs, credentials, and audit for a region; typically one per region per cloud.
2. Managed: Databricks owns file lifecycle (DROP deletes data); external: you own the location.
3. A view stays live and centrally governed; a copy drifts and multiplies the PII surface.
4. `SELECT` on the Gold schema (or specific tables/views) granted to the analyst group.
5. `system.access.audit` (with lineage in `system.access.table_lineage/column_lineage`).

Week 9:

1. Re-plans joins, partitions, and skew handling from runtime statistics instead of compile-time guesses.
2. When the "small" side no longer fits in executor memory, or AQE statistics were wrong.
3. A Python UDF serializes rows to a Python process per call; pandas UDFs move columnar Arrow batches.
4. Salting splits the hot key across partitions; the other side must be replicated per salt value.
5. The stage summary's shuffle read/write columns and the task-duration tail.

Week 10:

1. Automatic dependency resolution, incrementalization, retries, and quality metrics per dataset.
2. `expect_or_fail` for PII: bad PII should stop the pipeline, not silently pass or drop.
3. It records expectations, lineage, and per-run quality — your evidence base for audits.
4. When logic or schema changes invalidate incremental state (or for a deliberate rebuild).
5. Triggered: you pay only for the run; continuous bills idle waiting time.

Week 11:

1. Notification mode scales to millions of files where directory listing becomes the bottleneck.
2. The checkpoint records processed files/offsets; restarts resume, never repeat or skip.
3. Rows that do not match the schema; better than failing because the pipeline survives and you can reprocess.
4. Type 1 answers “what is true now”; Type 2 answers “what was true when”.
5. It orders CDC events by a sequence/timestamp column so late updates apply in the right order.

Week 12:

1. Failure isolation: one task’s crash or OOM does not strand sibling tasks.
2. Task values — set with `dbutils.jobs.taskValues.set` during a run.
3. Only the failed and downstream-dependent tasks — cheaper and faster than a full rerun.
4. Between Silver validation and Gold publish, as a blocking task.
5. They route failure paths inside the job graph instead of bolting on external monitors.

Week 13:

1. It stores offsets and state; without it a restart reprocesses or loses data.
2. Update mode — append forbids changing existing aggregate rows.
3. Batch-like execution of a streaming query: process all available data, then stop.
4. State schema is checkpointed; incompatible changes fail the restart by design.
5. Input rate vs. processing rate (and the batch duration trend).

Week 14:

1. State for windows/keys older than the watermark horizon — bounding memory.
2. The checkpoint recovers offsets/state; the sink must ignore re-delivered writes (idempotent MERGE/keys).
3. Overlapping windows — each event lands in multiple windows.
4. When state outgrows executor memory and spills must go to local disk/RocksDB.
5. Deduplication state must remember every seen key forever without a watermark.

Week 15:

1. Brokers acknowledge writes, consumers track offsets; redelivery is possible — dedupe at the sink.
2. Executor overload and memory pressure from oversized micro-batches.
3. Schemas evolve under contract instead of silently breaking deserialization.
4. One Kafka partition maps to one Spark task path — parallelism is partition-bound.
5. Event Hubs exposes a Kafka-compatible endpoint, so Kafka clients work unchanged.

Week 16:

1. Streaming DataFrames cannot MERGE directly; `foreachBatch` runs a batch MERGE per micro-batch with the checkpoint preserving exactly-once.

2. Delta's optimistic concurrency resolves stream/batch commit conflicts; partition discipline keeps them disjoint.
3. Sub-second or a few seconds end-to-end; otherwise a 1-minute trigger is far cheaper.
4. Streams drift from business truth (late data, replays); the batch path is the reconciler of record.
5. Continuous pays for always-on compute; triggered pays only when processing.

Week 17:

1. Environment, signatures, input examples, and standard metrics — the reproducibility scaffolding.
2. Governance: lineage, access control, and sharing live with the data, not per-workspace.
3. The alias (e.g. `champion`) — promotion moves the alias without redeploying jobs.
4. Logged code version, environment, data snapshot, and seed.
5. Otherwise you measure data drift, not model quality.

Week 18:

1. Using information unavailable at prediction time — e.g. "days_to_refund" in a churn model trained before refund events exist.
2. The lookup joins each label row only to feature rows with timestamps $\leq$ the label's event time.
3. So every model records exactly which feature tables and versions produced it.
4. The online store (low-latency lookups); the offline store serves training.
5. Skew is train-time features computed differently from serve-time; one governed feature pipeline feeds both.

Week 19:

1. Delta remains the governed source of truth; the index is a derived, rebuildable projection.
2. Delta-sync — the index updates automatically as the table changes.
3. Small chunks improve precision but lose context; large chunks dilute relevance.
4. Logged metrics make quality regressions visible and comparable across prompt/index changes.
5. Before embedding/indexing — never let PII reach the vector store or the model context.

Week 20:

1. Request/response payloads — enabling drift and quality monitoring without app changes.
2. Promotion changes the alias, not the job code — rollback is re-pointing the alias.
3. Partition-scoped `replaceWhere` (or key-based MERGE) so reruns replace rather than duplicate.
4. Retraining on drift does not guarantee better; promotion needs human or metric gates.
5. Provisioned concurrency for latency-critical; scale-to-zero for intermittent batch-adjacent calls.

Week 21:

1. People leave and rotate; service principals outlive individuals and are scoped to the job.
2. Joiner/mover/leaver group synchronization from the IdP — no orphaned access.

3. Key Vault-backed: your cloud vault controls rotation and policy; Databricks-backed is simpler but platform-managed.
4. Credentials already stolen and used from an allowed network — it is a perimeter, not authentication.
5. Data-plane traffic leaves the public internet; exposure shifts to private-endpoint and DNS hygiene.

Week 22:

1. You enumerate every downstream table/dashboard consuming a column before changing it.
2. `system.access.audit`.
3. Sharing serves data through the protocol's short-lived pre-signed access — no standing credentials cross the boundary.
4. Access ends; recipients cannot refresh or re-read the shared data.
5. Non-Databricks recipients consume via open connectors (pandas, Spark, Power BI) without a Databricks account.

Week 23:

1. Invalid resource definitions, missing variables, and bad references — before any cloud change.
2. Targets isolate workspaces, variables, and deployment state; flags invite accidental prod edits.
3. Concurrent applies corrupt local state; remote locked state enables team/CI collaboration.
4. `system.billing.usage`.
5. Auto-termination/auto-stop and job clusters (or serverless) instead of always-on all-purpose compute.

Week 24:

1. Yours to answer — name the domain and the lab you will rebuild.
2. Run the clean-clone redeploy drill; a platform that cannot redeploy is not finished.
3. Cost per month, p95 latency, and rows processed per day.
4. Yours to answer — write it in the README's lessons-learned section.
5. Practice the one-sentence-per-box drill before the review.

11 Appendix E: Datasets & Project Data

Prefer data already available inside Databricks (`/databricks-datasets/` in the workspace file store, or the `samples` catalog) — it sits next to your compute, so no egress cost or download step.

- **NYC Taxi & Limousine Trip Records** (`/databricks-datasets/nyctaxi/`, also <https://registry.opendata.aws/nyc-tlc-trip-records-pds/>): billions of trip rows in CSV/Parquet. Used in Week 4 Spark tuning, the Week 5 optimization benchmark, and the Milestone 2 analytics capstone.
- **samples catalog** (`samples.tpch`, `samples.nyctaxi`): governed sample schemas present in every Unity Catalog workspace — ideal for SQL warehouse and Genie labs in Weeks 6–7.

- **Olist Brazilian E-Commerce dataset** (<https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>) 100k orders across customers, sellers, products, payments, and reviews — a natural star schema. Used in the Milestone 1 and Milestone 3 capstones; upload the CSVs to a Unity Catalog volume.

Synthetic data: generate customers, orders, clickstream, and card transactions with Python Faker (<https://github.com/joke2k/faker>); the Week 15 Kafka producer script shows the pattern for streaming events. Expected volumes per milestone: Milestone 1 ~1M rows across Bronze–Gold; Milestone 2 ~10M rows optimized Delta; Milestone 3 ~50M raw events per batch; Milestone 4 ~5k events/minute sustained; Milestone 5 ~100k feature rows; Milestone 6 small governed samples (~100k rows) with injected PII.

12 Appendix F: Portfolio & Presentation Guide

Each capstone should ship as a public GitHub repository you can walk an interviewer through.

README structure (in this order):

1. **Problem:** one paragraph — who hurts, at what scale, and what “done” means.
2. **Architecture diagram first:** reviewers decide in 10 seconds; lead with the picture.
3. **Setup:** exact commands to reproduce from a clean environment (prereqs, bundle deploy, seed data).
4. **Screenshots:** working dashboard, pipeline run, and one alert firing.
5. **Cost estimate:** monthly DBU table with the two optimization decisions you made.
6. **Lessons learned:** one failure you hit and how you fixed it.

Resume bullet templates (two per milestone — fill in the brackets):

- **M1:** Built a medallion lakehouse on Databricks (Bronze/Silver/Gold Delta tables) with quarantine accounting reconciling [N] rows/day and time-travel recovery drills.
- **M1:** Tuned PySpark workloads using Spark UI evidence, cutting a [N] TB nightly job from [X] hours to [Y] minutes.
- **M2:** Built a governed analytics layer (liquid-clustered Delta, serverless SQL warehouse, AI/BI + Genie) serving [N] analysts under Unity Catalog PII masking.
- **M2:** Benchmarked Z-ORDER vs. liquid clustering, documenting a layout decision saving [X]% on query latency at [N] rows.
- **M3:** Automated a batch ELT platform with Auto Loader, DLT expectations, and a multi-task Workflow with repair-run recovery, eliminating duplicate loads on re-runs.
- **M3:** Implemented SCD Type 2 CDC with AUTO CDC, preserving full customer history across [N] daily extracts.
- **M4:** Built a real-time fraud-detection stream (Kafka → Structured Streaming windows → Delta → AI/BI) flagging impossible travel within 10 minutes.
- **M4:** Implemented exactly-once streaming with checkpoints and idempotent `foreachBatch MERGE`, sustaining [N] events/minute with zero duplicate alerts on replay.
- **M5:** Productionized MLOps: point-in-time feature tables → MLflow tracking → Unity Catalog registry → serverless serving with Lakehouse Monitoring drift alarms.
- **M5:** Built a RAG assistant with Mosaic AI Vector Search over [N] documents, evaluating quality with MLflow metrics before launch.

- **M6:** Deployed a governed data platform as code with Databricks Asset Bundles, Terraform, and GitHub Actions dev→staging→prod promotion.
- **M6:** Built per-domain chargeback reporting from `system.billing.usage`, cutting monthly DBU spend [X]% via auto-termination and serverless policies.

Talking about your project in interviews: open with the problem and its scale (“10M rows nightly, 500 analysts”), then show the architecture diagram and narrate two design decisions with their trade-offs — interviewers score reasoning, not service names. Have one failure story ready (what broke, how you detected it, what you changed). Close with numbers: DBU cost per month, latency, row counts. If asked “what would you do differently,” you pass only if you have a real answer — write it in the README’s lessons-learned section while the project is fresh.

13 6-Month Progress Tracking Timetable

Week	Primary Focus	Key Project / Capstone	Status
1	Databricks Platform & Workspace	CLI + SDK smoke test, governed compute policy	☐
2	Delta Lake Foundations	Schema enforcement/evolution & RESTORE drill	☐
3	Medallion Architecture	Bronze/Silver/Gold Olist flow with quarantine	☐
4	Apache Spark on Databricks	Cap 1: Retail Medallion Lakehouse	☐
5	Delta Optimization	OPTIMIZE/Z-ORDER vs. liquid clustering benchmark	☐
6	Databricks SQL Warehouses	Serverless warehouse + materialized view	☐
7	AI/BI Dashboards & Genie	Executive dashboard + instructed Genie space	☐
8	Unity Catalog Governance	Cap 2: Governed Analytics Lakehouse	☐
9	PySpark Tuning	Skew/shuffle remediation with Spark UI evidence	☐
10	Delta Live Tables	DLT pipeline with expectations & event log	☐
11	Auto Loader & CDC	Incremental ingestion + SCD2 customer history	☐
12	Workflows Orchestration	Cap 3: Orchestrated Batch ELT Platform	☐
13	Structured Streaming	Checkpointed stream with restart proof	☐
14	State, Watermarks & Exactly-Once	Watermarked dedup with RocksDB comparison	☐
15	Kafka & Event Hubs Integration	Kafka→Delta with replay reconciliation	☐
16	Real-Time Lakehouse	Cap 4: Real-Time Fraud Telemetry	☐
17	MLflow & Tracking	Six autologged runs + champion alias	☐
18	Feature Store	Point-in-time training set with lineage	☐
19	Mosaic AI & RAG	Vector Search RAG assistant with MLflow eval	☐
20	Serving & MLOps	Cap 5: End-to-End MLOps Pipeline	☐

Week	Primary Focus	Key Project / Capstone	Status
21	Security, IAM & Network	Service principal + secret scope hardening	☐
22	Lineage, Audit & Delta Sharing	Column lineage report + share revocation test	☐
23	Asset Bundles, Terraform & FinOps	Bundle CI/CD with staging gate + DBU report	☐
24	Final Capstone & Certification Prep	Cap 6: Governed Data Platform + 2 mock exams	☐

"Consistency is the architecture of mastery."

Best of luck on your 6-month journey to becoming a Master Databricks Data Engineer!